

Aula 14: Tuplas e Dicionários

Luís Felipe

UFF

17 de Novembro de 2025

Tuplas

- Tuplas são similares a listas, isto é, uma sequência de dados de qualquer tipo.
- Porém, ao contrário de listas, as **tuplas são imutáveis**.
- Tuplas são representadas por uma sequência de valores separados por vírgula, e entre um abre e fecha **parênteses**.
 - ▶ (1,2,5,"aaa") é uma tupla de 4 elementos.
- As operações para acessar os elementos ou subsequências de uma lista e de uma string, também funcionam em tuplas.

```
>>> a = (1,2,5, 'aaa')
```

```
>>> a[2]
```

```
5
```

```
>>> a[1:3]
```

```
(2, 5)
```

Empacotamento e desempacotamento

- Os elementos de uma tupla podem ser acessados de uma forma implícita na atribuição, **desempacotamento**

```
>>> x,y = (9,10)
```

```
>>> x
```

```
9
```

```
>>> y
```

```
10
```

```
>>>
```

- A tupla pode ser implicitamente criada apenas separando por vírgula, **empacotamento**

```
>>> 9,10
```

```
(9,10)
```

Tuplas

- Como strings, tuplas são imutáveis

```
>>> a = (1,2,5"aa")
```

```
>>> a[2]=9
```

```
Traceback (most recent call last):
```

```
File "<python-input-235>", line 1, in <module>
```

```
    a[2] = 9
```

```
    ~^^^
```

```
TypeError: 'tuple' object does not support item assignment
```

- Qual utilidade, então? Veremos hoje ao falar de **dicionários**.

Dicionários

- Dicionários são estruturas de dados que associam uma **chave** com um **valor**.
- Os valores podem ser um dado de qualquer tipo, mas as chaves só podem ser dados de tipos **imutáveis**.
- As **chaves** precisam ser **únicas**.
- Exemplo de criação de dicionário:

```
1 >>> dd = {" Jose": 12345678, " maria": 78765432}
2 >>> type(dd)
3 <class 'dict'>
```

- O dicionário acima pode representar uma agenda de telefones, com um nome como uma **chave** (uma string, que é imutável) e o **valor associado a cada chave** é o telefone (um inteiro).

```
1 >>> dd[" maria"]
2 78765432
```

Dicionários: atualização e inserção

O **valor** associado a uma chave pode ser **modificado**.

Também é possível **incluir** uma **nova chave (com seu valor)** no dicionário.

```
1 dd = {"Jose": 12345678, "maria": 78765432}
2 print(dd)
3 # {'Jose': 12345678, 'maria': 78765432}
4
5 dd["maria"] = 777777          # altera valor existente
6 dd["carlos"] = 888888        # inclui nova chave
7
8 print(dd)
9 # {'carlos': 888888, 'maria': 777777, 'Jose': 12345678}
```

Obs.: Formas de criar um dicionário vazio:

```
1 nome_dicionario = {}
```

```
1 nome_dicionario = dict()
```

Dicionários: ordem das chaves

A ordem dos pares *chave:valor* pode variar conforme inserções/alterações (no Python 3.7+ a ordem de inserção é preservada, mas não deve ser usada como regra de lógica do programa).

```
1 dd["ana"] = 9999999
2
3 print(dd)
4 # {'carlos': 888888, 'maria': 777777, 'Jose': 12345678, 'ana': 9999999}
```

Operações em Dicionários

O laço `for` aplicado a um dicionário percorre todas as chaves (na ordem interna).

```
1 for x in dd:  
2     print(x)  
3  
4 # Saída:  
5  
6 # carlos  
7 # maria  
8 # Jose  
9 # ana
```

O operador `in` verifica se uma chave está no dicionário.

```
1 'ana' in dd  
2 # True  
3  
4 'Ana' in dd  
5 # False  
6  
7 dd['zico']  
8 # KeyError: 'zico'
```

Dicionários: acesso e método items

O método `items()` retorna tuplas (chave, valor):

```
1 dd.items()
2
3 # dict_items([
4 #   ('ana', 333444555),
5 #   ('maria', 77777777),
6 #   ('Jose', 12345678),
7 #   ('carlos', 1112223334)
8 # ])
```

Exemplo: Letra mais comum

Objetivo: função que, dada uma string, retorna a letra mais comum. Se houver empate, retorna qualquer uma das mais frequentes.

Ideia: usar um dicionário para contar cada letra. A letra será a chave; o valor será o número de ocorrências.

```
1 def contalettra(s):
2     conta = {}      # dicionário vazio. Outra forma de criar: conta = dict()
3     for car in s:
4         if car in conta:
5             conta[car] += 1
6         else:
7             conta[car] = 1
8     # Ao final: dicionário com pares letra/contador
9
10    s = input()
11    print(contalettra(s))
```

```
1 olaeusuousoueusou
2 {'o': 4, 'l': 1, 'a': 1, 'e': 2, 'u': 6, 's': 3}
```

Determinando a letra mais comum

Continuação da função `conta letra`:

```
1 def conta letra(s):  
2     conta = {}  
3     for car in s:  
4         if car in conta:  
5             conta[car] += 1  
6         else:  
7             conta[car] = 1  
8  
9     letrama is = ''  
10    for x in conta:  
11        if letrama is == '': # nenhuma mais comum ainda  
12            letrama is = x  
13        elif conta[x] > conta[letrama is]:  
14            letrama is = x  
15  
16    return letrama is
```

Função completa e testes

Função **conta letra** completa e exemplos de uso:

```
1 def conta letra(s):
2     conta = {}
3     for car in s:
4         if car in conta:
5             conta[car] += 1
6         else:
7             conta[car] = 1
8     letrama is = ''
9     for x in conta:
10        if letrama is == '':
11            letrama is = x
12        elif conta[x] > conta[letrama is]:
13            letrama is = x
14    return letrama is, conta[letrama is]
15
16
17 print(conta letra("ouviram do ipiranga"))
18 # ('i', 3)
19
20 print(conta letra("ouviram do ipirangaaaa"))
21 # ('a', 6)
```

Funções dict(), del() e len()

`dict()` cria um dicionário vazio.

```
1 pares = dict()    # ou pares = {}  
2 print(pares)      # {}
```

Para adicionar um par `chave:valor`:

```
1 pares[13] = "Valor da Sorte"  
2 print(pares)      # {13: 'Valor da Sorte'}
```

A função `del` remove um par do dicionário:

```
1 pares = {51: "Boa ideia", 13: "Valor da Sorte", 31: "Dia da Sorte"}  
2 del pares[13]      # ou pares.pop(13)  
3 print(pares)  
4 # {51: 'Boa ideia', 31: 'Dia da Sorte'}
```

A função `len()` retorna o tamanho do dicionário:

```
1 print(len(pares))  # 2
```

Lendo nomes e telefones

O programa abaixo lê 5 pares **nome:telefone** e armazena no dicionário.

```
1 pares = dict() # ou pares = {}  
2 for i in range(5):  
3     nome = input("Digite nome: ")  
4     fone = input("Digite o telefone de " + nome + ": ")  
5     pares[nome] = fone  
6     print(pares)
```

Criando dicionário diretamente

Podemos criar o dicionário já com conteúdo:

```
1 pares = {  
2     "Maria": "3456-7922",  
3     "Ana": "3214-2211",  
4     "Giovanna": "4564-1234"  
5 }  
6 print(pares)
```

Visualizações sobre dicionário

`d.items()`: retorna todos os pares (chave, valor).

`d.keys()`: retorna todas as chaves.

`d.values()`: retorna todos os valores.

```
1 pares.items()  
2 pares.keys()  
3 pares.values()
```

Tuplas como chaves e métodos `items()`, `keys()`, `values()`

```
1 # (cidade, dia) -> temperatura
2 temps = {
3     ("Rio de Janeiro", "Segunda"): 32,
4     ("Rio de Janeiro", "Terça"): 30,
5     ("São Paulo", "Segunda"): 25,
6     ("São Paulo", "Terça"): 27
7 }
8 print(temps.items())
9 # dict_items([
10 #     (("Rio de Janeiro", "Segunda"), 32),
11 #     (("Rio de Janeiro", "Terça"), 30),
12 #     (("São Paulo", "Segunda"), 25),
13 #     (("São Paulo", "Terça"), 27)
14 # ])
15 print(temps.keys())
16 # dict_keys([
17 #     ("Rio de Janeiro", "Segunda"),
18 #     ("Rio de Janeiro", "Terça"),
19 #     ("São Paulo", "Segunda"),
20 #     ("São Paulo", "Terça")
21 # ])
22 print(temps.values())
23 # dict_values([32, 30, 25, 27])
```

Iterando sobre chaves e valores

Percorrendo chaves:

```
1 for chave in pares:  
2     print(chave, ":", pares[chave])
```

Percorrendo pares:

```
1 for chave, valor in pares.items():  
2     print(chave, ":", valor)
```

Percorrendo chaves ordenadas:

```
1 for chave in sorted(pares):  
2     print(chave, ":", pares[chave])  
3  
4 for chave in sorted(pares.keys()):  
5     print(chave, ":", pares[chave])
```

Percorrendo apenas valores:

```
1 for valor in pares.values():  
2     print(valor)
```

Consultando Dicionários

Consultar diretamente uma chave inexistente gera `KeyError`:

```
1 pares = {51: "Boa ideia", 13: "Valor da Sorte", 31: "Dia do Azar"}  
2 print(pares[51]) # Boa ideia  
3 print(pares[52]) # KeyError: 52
```

Para evitar o erro, podemos usar a função `get()`.

Função `get()`

A função `get(chave)` retorna `None` se a chave não existir; caso contrário, retorna o valor associado.

```
1 pares = {51: "Boa ideia", 13: "Valor da Sorte", 31: "Dia do Azar"}  
2 print(pares.get(13)) # Valor da Sorte
```

A função `get(chave, default)` retorna `default` se a chave não existir.

```
1 pares = {}  
2 print(pares.get(44)) # None  
3  
4 pares = {51: "Boa ideia", 13: "Valor da Sorte", 31: "Dia do Azar"}  
5 print(pares.get(22, "Vazia")) # Vazia  
6 print(pares.get(51, "Vazia")) # Boa ideia
```

Uso de Dicionário como Matriz Esparsa

Matriz esparsa: armazena apenas posições não nulas.

Exemplo de função para exibir matriz armazenada em dicionário:

```
1 def mostra(vs):                                     # exibe a matriz organizada
2     qtdLinhas = vs["Número de Linhas"]
3     qtdColunas = vs["Número de Colunas"]
4     for linha in range(qtdLinhas):
5         for coluna in range(qtdColunas):
6             valor = vs.get((linha, coluna), 0)
7             print("%4d" % valor, end=" ")          # 4 espaços para cada valor
8         print()
9     return
```

Matriz Esparsa: Programa Principal

Exemplo de uso:

```
1 valores = {}  
2 valores["Número de Linhas"] = 5  
3 valores["Número de Colunas"] = 10  
4  
5 valores[(1, 1)] = 13  
6 valores[(0, 8)] = -4  
7 valores[(4, 7)] = 75  
8  
9 print(valores)  
10 mostra(valores)
```

Saída esperada:

{'Número de Linhas': 5, 'Número de Colunas': 10, (1, 1): 13, (0, 8): -4, (4, 7): 75}

0	0	0	0	0	0	0	0	-4	0
0	13	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	75	0	0

Utilidade de Tuplas em Dicionários

Por que usar tuplas como chaves?

- Tuplas são **imutáveis**, logo podem ser usadas como chaves (listas não podem).
- Permitem criar **chaves compostas**, representando múltiplos atributos.
- Úteis para representar **coordenadas**, **pares de valores** ou parâmetros.

Exemplos:

```
1 # Coordenadas em uma matriz esparsa
2 matriz = {}
3 matriz[(2, 3)] = 10
4 matriz[(0, 1)] = 5
5
6 # Distâncias entre cidades
7 distancias = {}
8 distancias[("Rio de Janeiro", "São Paulo")] = 430
9 distancias[("Rio de Janeiro", "Belo Horizonte")] = 440
10
11 # Cache de função (memorização)
12 cache = {}
13 def soma(a, b):
14     if (a, b) not in cache:
15         cache[(a, b)] = a + b
16     return cache[(a, b)]
```